```
// ============================================
// PAYCHECK BUDDY — APP LOGIC
// ============================================

// ============================================
// STATE
// ============================================

let appState = {
  user: {
    name: '',
    mode: 'solo', // 'solo' or 'household'
    onboarded: false
  },
  paycheck: {
    hourlyRate: 0,
    regularHours: 0,
    overtimeHours: 0,
    frequency: 'biweekly',
    filingStatus: 'single',
    dependents: 0,
    state: '',
    healthInsurance: 0,
    retirement: 0,
    takeHome: 0,
    grossPay: 0
  },
  bills: [],
  pods: [],
  receipts: [],
  settings: {
    darkMode: false,
    tipsEnabled: true,
    quietMode: false
  }
};

// ============================================
// TAX DATA
// ============================================

const STATE_TAX_RATES = {
  'AL': { name: 'Alabama', rate: 0.05, hasLocal: false },
  'AK': { name: 'Alaska', rate: 0.00, hasLocal: false },
```

```
'AZ': { name: 'Arizona', rate: 0.025, hasLocal: false },
'AR': { name: 'Arkansas', rate: 0.055, hasLocal: false },
'CA': { name: 'California', rate: 0.093, hasLocal: false },
'CO': { name: 'Colorado', rate: 0.044, hasLocal: false },
'CT': { name: 'Connecticut', rate: 0.065, hasLocal: false },
'DE': { name: 'Delaware', rate: 0.066, hasLocal: false },
'FL': { name: 'Florida', rate: 0.00, hasLocal: false },
'GA': { name: 'Georgia', rate: 0.055, hasLocal: false },
'HI': { name: 'Hawaii', rate: 0.08, hasLocal: false },
'ID': { name: 'Idaho', rate: 0.058, hasLocal: false },
'IL': { name: 'Illinois', rate: 0.0495, hasLocal: false },
'IN': { name: 'Indiana', rate: 0.0315, hasLocal: false },
'IA': { name: 'Iowa', rate: 0.06, hasLocal: false },
'KS': { name: 'Kansas', rate: 0.057, hasLocal: false },
'KY': { name: 'Kentucky', rate: 0.045, hasLocal: false },
'LA': { name: 'Louisiana', rate: 0.0425, hasLocal: false },
'ME': { name: 'Maine', rate: 0.075, hasLocal: false },
'MD': { name: 'Maryland', rate: 0.0575, hasLocal: true, localRate: 0.0320 },
'MA': { name: 'Massachusetts', rate: 0.05, hasLocal: false },
'MI': { name: 'Michigan', rate: 0.0425, hasLocal: false },
'MN': { name: 'Minnesota', rate: 0.0785, hasLocal: false },
'MS': { name: 'Mississippi', rate: 0.05, hasLocal: false },
'MO': { name: 'Missouri', rate: 0.054, hasLocal: false },
'MT': { name: 'Montana', rate: 0.069, hasLocal: false },
'NE': { name: 'Nebraska', rate: 0.0664, hasLocal: false },
'NV': { name: 'Nevada', rate: 0.00, hasLocal: false },
'NH': { name: 'New Hampshire', rate: 0.00, hasLocal: false },
'NJ': { name: 'New Jersey', rate: 0.0637, hasLocal: false },
'NM': { name: 'New Mexico', rate: 0.059, hasLocal: false },
'NY': { name: 'New York', rate: 0.0685, hasLocal: true, localRate: 0.0346 },
'NC': { name: 'North Carolina', rate: 0.0499, hasLocal: false },
'ND': { name: 'North Dakota', rate: 0.0290, hasLocal: false },
'OH': { name: 'Ohio', rate: 0.0399, hasLocal: true, localRate: 0.02 },
'OK': { name: 'Oklahoma', rate: 0.0475, hasLocal: false },
'OR': { name: 'Oregon', rate: 0.099, hasLocal: false },
'PA': { name: 'Pennsylvania', rate: 0.0307, hasLocal: true, localRate: 0.01 },
'RI': { name: 'Rhode Island', rate: 0.0599, hasLocal: false },
'SC': { name: 'South Carolina', rate: 0.065, hasLocal: false },
'SD': { name: 'South Dakota', rate: 0.00, hasLocal: false },
'TN': { name: 'Tennessee', rate: 0.00, hasLocal: false },
'TX': { name: 'Texas', rate: 0.00, hasLocal: false },
'UT': { name: 'Utah', rate: 0.0485, hasLocal: false },
'VT': { name: 'Vermont', rate: 0.0875, hasLocal: false },
'VA': { name: 'Virginia', rate: 0.0575, hasLocal: false },
'WA': { name: 'Washington', rate: 0.00, hasLocal: false },
'WV': { name: 'West Virginia', rate: 0.065, hasLocal: false },
'WI': { name: 'Wisconsin', rate: 0.0765, hasLocal: false },
```

```
  'WY': { name: 'Wyoming', rate: 0.00, hasLocal: false },
  'DC': { name: 'Washington D.C.', rate: 0.0850, hasLocal: false }
};

// 2024 Federal Tax Brackets (per pay period annualized)
const FEDERAL_BRACKETS = {
  single: [
    { min: 0, max: 11600, rate: 0.10 },
    { min: 11600, max: 47150, rate: 0.12 },
    { min: 47150, max: 100525, rate: 0.22 },
    { min: 100525, max: 191950, rate: 0.24 },
    { min: 191950, max: 243725, rate: 0.32 },
    { min: 243725, max: 609350, rate: 0.35 },
    { min: 609350, max: Infinity, rate: 0.37 }
  ],
  married: [
    { min: 0, max: 23200, rate: 0.10 },
    { min: 23200, max: 94300, rate: 0.12 },
    { min: 94300, max: 201050, rate: 0.22 },
    { min: 201050, max: 383900, rate: 0.24 },
    { min: 383900, max: 487450, rate: 0.32 },
    { min: 487450, max: 731200, rate: 0.35 },
    { min: 731200, max: Infinity, rate: 0.37 }
  ],
  head: [
    { min: 0, max: 16550, rate: 0.10 },
    { min: 16550, max: 63100, rate: 0.12 },
    { min: 63100, max: 100500, rate: 0.22 },
    { min: 100500, max: 191950, rate: 0.24 },
    { min: 191950, max: 243700, rate: 0.32 },
    { min: 243700, max: 609350, rate: 0.35 },
    { min: 609350, max: Infinity, rate: 0.37 }
  ]
};

const PAY_PERIODS = {
  weekly: 52,
  biweekly: 26,
  semimonthly: 24,
  monthly: 12
};

// ============================================
// BUDDY TIPS
// ============================================

const DAILY_TIPS = [
```

```
  "Try the 50/30/20 rule: 50% needs, 30% wants, 20% savings! 💡",
  "Even saving $5 a week adds up to $260 a year! 🎯",
  "Pay yourself first — put savings aside before spending! 💰",
  "Review your subscriptions — you might be paying for things you forgot about! 📱
  "An emergency fund of 3 months expenses is the goal! 🛡️",
  "Cooking at home just 3 more times a week can save hundreds a month! 🍳",
  "Check if your employer offers 401k matching — that's free money! 💎",
  "Round up your bill payments — the extra goes to principal! 📈",
  "A budget isn't a restriction — it's a plan for your money! 📋",
  "Track every dollar for one week and you'll be amazed what you find! 👀",
  "Gas prices vary — apps like GasBuddy can save you real money! ⛽",
  "Automate your savings so you never have to think about it! 🤖",
  "Before a big purchase, wait 48 hours. Still want it? Then it's worth it! ⏳",
  "Your biggest financial asset is your income — protect it! 💪",
  "Small wins count — celebrate every bill you pay off! 🎉"
];

// ============================================
// INIT
// ============================================

document.addEventListener('DOMContentLoaded', () => {
  loadData();
  populateStateDropdown();

  // Show splash, then route to correct screen
  setTimeout(() => {
    hideSplash();
  }, 2500);
});

function hideSplash() {
  const splash = document.getElementById('splash-screen');
  splash.style.opacity = '0';
  splash.style.transition = 'opacity 0.5s ease';

  setTimeout(() => {
    splash.style.display = 'none';

    if (!appState.user.onboarded) {
      showOnboarding();
    } else {
      showMainApp();
    }
  }, 500);
}
```

```
function showOnboarding() {
  document.getElementById('onboarding').style.display = 'flex';
}

function showMainApp() {
  document.getElementById('main-app').style.display = 'flex';
  updateHomeScreen();
  updateSettingsScreen();
  renderBills();
  renderPods();
}

// ============================================
// ONBOARDING
// ============================================

let currentStep = 1;

function nextStep() {
  const current = document.querySelector(`.onboard-step[data-step="${currentStep}
  currentStep++;
  const next = document.querySelector(`.onboard-step[data-step="${currentStep}"]`

  if (current) current.classList.remove('active');
  if (next) next.classList.add('active');
}

function saveName() {
  const name = document.getElementById('user-name').value.trim();
  if (!name) {
    shakeElement(document.getElementById('user-name'));
    return;
  }
  appState.user.name = name;

  // Update greeting in next step
  document.getElementById('mode-greeting').textContent =
    `Nice to meet you, ${name}! Are we managing money just for you or your whole

  nextStep();
}

function selectMode(mode) {
  appState.user.mode = mode;

  // Update welcome message
  const msg = mode === 'household'
```

```
    ? `Let's get it, ${appState.user.name}! Your household's money finally has a
    : `Let's get it, ${appState.user.name}! Your money finally has a plan! 🎉`;
  document.getElementById('welcome-message').textContent = msg;

  nextStep();
  setTimeout(() => launchConfetti(), 300);
}

function finishOnboarding() {
  appState.user.onboarded = true;
  saveData();

  document.getElementById('onboarding').style.opacity = '0';
  document.getElementById('onboarding').style.transition = 'opacity 0.4s';

  setTimeout(() => {
    document.getElementById('onboarding').style.display = 'none';
    showMainApp();
  }, 400);
}

// ============================================
// NAVIGATION
// ============================================

function switchTab(tab) {
  // Haptic
  triggerHaptic();

  // Hide all screens
  document.querySelectorAll('.tab-screen').forEach(s => s.classList.remove('activ
  document.querySelectorAll('.nav-btn').forEach(b => b.classList.remove('active')

  // Show selected
  document.getElementById(`tab-${tab}`).classList.add('active');
  document.getElementById(`nav-${tab}`).classList.add('active');

  // Scroll content to top
  document.getElementById('screen-content').scrollTop = 0;

  // Tab-specific updates
  if (tab === 'home') updateHomeScreen();
  if (tab === 'bills') renderBills();
  if (tab === 'pods') renderPods();
}

// ============================================
```

```
// PAYCHECK CALCULATOR
// ============================================

function populateStateDropdown() {
  const select = document.getElementById('state-select');
  if (!select) return;

  Object.entries(STATE_TAX_RATES)
    .sort((a, b) => a[1].name.localeCompare(b[1].name))
    .forEach(([code, data]) => {
      const option = document.createElement('option');
      option.value = code;
      option.textContent = data.rate === 0
        ? `${data.name} (No State Tax) 🎉`
        : `${data.name} (${(data.rate * 100).toFixed(1)}%)`;
      select.appendChild(option);
    });
}

function calculatePaycheck() {
  const hourlyRate = parseFloat(document.getElementById('hourly-rate')?.value) ||
  const regularHours = parseFloat(document.getElementById('regular-hours')?.value
  const overtimeHours = parseFloat(document.getElementById('overtime-hours')?.val
  const frequency = document.getElementById('pay-frequency')?.value || 'biweekly'
  const filingStatus = document.getElementById('filing-status')?.value || 'single
  const dependents = parseInt(document.getElementById('dependents')?.value) || 0;
  const stateCode = document.getElementById('state-select')?.value || '';
  const healthInsurance = parseFloat(document.getElementById('health-insurance')?
  const retirementPct = parseFloat(document.getElementById('retirement')?.value)

  if (hourlyRate === 0 || regularHours === 0) {
    document.getElementById('paycheck-result').textContent = '$0.00';
    return;
  }

  // Gross pay per period
  const regularPay = hourlyRate * regularHours;
  const overtimePay = hourlyRate * 1.5 * overtimeHours;
  const grossPay = regularPay + overtimePay;

  // Pre-tax deductions
  const retirementAmount = grossPay * (retirementPct / 100);
  const preTaxDeductions = healthInsurance + retirementAmount;
  const taxableIncome = grossPay - preTaxDeductions;

  // Annualize for bracket calculation
  const periods = PAY_PERIODS[frequency];
```

```
const annualTaxable = taxableIncome * periods;

// Dependent credit ($500 per dependent, $2000 for under 17 — simplified)
const dependentCredit = dependents * 500;

// Federal tax
const bracketKey = filingStatus === 'married' ? 'married' : filingStatus === 'h
const brackets = FEDERAL_BRACKETS[bracketKey];
const adjustedAnnual = Math.max(0, annualTaxable - dependentCredit);
let annualFederal = calculateBracketTax(adjustedAnnual, brackets);
const federalTax = annualFederal / periods;

// State tax
let stateTax = 0;
let localTax = 0;
if (stateCode && STATE_TAX_RATES[stateCode]) {
  const stateData = STATE_TAX_RATES[stateCode];
  stateTax = taxableIncome * stateData.rate;
  if (stateData.hasLocal) {
    localTax = taxableIncome * (stateData.localRate || 0);
  }
}

// FICA
const socialSecurity = Math.min(grossPay * 0.062, (160200 / periods) * 0.062);
const medicare = grossPay * 0.0145;

// Total deductions
const totalDeductions = federalTax + stateTax + localTax + socialSecurity + med
const takeHome = grossPay - totalDeductions;

// Save to state
appState.paycheck = {
  hourlyRate, regularHours, overtimeHours, frequency,
  filingStatus, dependents, state: stateCode,
  healthInsurance, retirement: retirementPct,
  grossPay, takeHome,
  federalTax, stateTax, localTax,
  socialSecurity, medicare,
  retirementAmount, preTaxDeductions
};

saveData();

// Update UI
animateNumber('paycheck-result', takeHome, true);
showBreakdown({
```

```
    grossPay, federalTax, stateTax, localTax,
    socialSecurity, medicare, healthInsurance,
    retirementAmount, takeHome
  }, stateCode);

  updateHomeScreen();
}

function calculateBracketTax(income, brackets) {
  let tax = 0;
  for (const bracket of brackets) {
    if (income <= 0) break;
    const taxable = Math.min(income - bracket.min, bracket.max - bracket.min);
    if (taxable > 0) {
      tax += taxable * bracket.rate;
    }
  }
  return Math.max(0, tax);
}

function showBreakdown(data, stateCode) {
  const card = document.getElementById('paycheck-breakdown');
  const list = document.getElementById('breakdown-list');

  const stateName = stateCode && STATE_TAX_RATES[stateCode]
    ? STATE_TAX_RATES[stateCode].name
    : 'State';

  const items = [
    { label: 'Gross Pay', value: data.grossPay, type: 'positive' },
    { label: 'Federal Income Tax', value: -data.federalTax, type: 'negative' },
    { label: `${stateName} Tax`, value: -data.stateTax, type: data.stateTax === 0
    { label: 'Local Tax', value: -data.localTax, type: data.localTax === 0 ? '' :
    { label: 'Social Security (6.2%)', value: -data.socialSecurity, type: 'negati
    { label: 'Medicare (1.45%)', value: -data.medicare, type: 'negative' },
    { label: 'Health Insurance', value: -data.healthInsurance, type: data.healthI
    { label: '401k Contribution', value: -data.retirementAmount, type: data.retir
    { label: '✅ Take-Home Pay', value: data.takeHome, type: 'total' },
  ];

  list.innerHTML = items.map(item => {
    if (item.value === 0 && item.type !== 'total' && item.type !== 'positive') re
    const displayValue = item.value < 0
      ? `-$${Math.abs(item.value).toFixed(2)}`
      : `$${item.value.toFixed(2)}`;
    return `
      <div class="breakdown-item ${item.type === 'total' ? 'total' : ''}">
```

```
        <span class="breakdown-label">${item.label}</span>
        <span class="breakdown-value ${item.type}">${displayValue}</span>
      </div>
    `;
  }).join('');

  card.style.display = 'block';
}

// ============================================
// HOME SCREEN
// ============================================

function updateHomeScreen() {
  const name = appState.user.name;
  const hour = new Date().getHours();
  const greeting = hour < 12 ? 'Good morning' : hour < 17 ? 'Good afternoon' : 'G

  const greetEl = document.getElementById('home-greeting');
  if (greetEl) {
    greetEl.textContent = name
      ? `${greeting}, ${name}! 👋`
      : `${greeting}! 👋`;
  }

  // Calculate totals
  const takeHome = appState.paycheck.takeHome || 0;
  const totalBills = appState.bills.reduce((sum, b) => sum + b.amount, 0);
  const totalPods = appState.pods.reduce((sum, p) => sum + (p.contribution || 0),
  const safeToSpend = Math.max(0, takeHome - totalBills - totalPods);

  // Update stats
  animateNumber('stat-takehome', takeHome, true);
  animateNumber('stat-bills', totalBills, true);
  animateNumber('stat-saving', totalPods, true);
  animateNumber('safe-spend-amount', safeToSpend, true);

  // Update Buddy expression based on financial health
  updateBuddyExpression(takeHome, totalBills, totalPods, safeToSpend);

  // Update daily tip
  updateDailyTip();

  // Update setup checklist
  updateSetupChecklist(takeHome, totalBills, totalPods);

  // Update payday countdown
```

```
  updatePaydayCountdown();
}

function updateBuddyExpression(takeHome, bills, pods, safe) {
  const buddyImg = document.getElementById('home-buddy-img');
  if (!buddyImg) return;

  let expression = 'buddy-neutral';

  if (takeHome === 0) {
    expression = 'buddy-confused';
  } else if (safe > takeHome * 0.3) {
    expression = 'buddy-happy';
  } else if (safe > takeHome * 0.1) {
    expression = 'buddy-neutral';
  } else if (safe > 0) {
    expression = 'buddy-sad';
  } else {
    expression = 'buddy-broke';
  }

  buddyImg.src = `images/${expression}.png`;
}

function updateDailyTip() {
  const tipEl = document.getElementById('home-tip');
  if (!tipEl || !appState.settings.tipsEnabled) return;

  const dayIndex = new Date().getDate() % DAILY_TIPS.length;
  tipEl.innerHTML = `<p>${DAILY_TIPS[dayIndex]}</p>`;
}

function updateSetupChecklist(takeHome, bills, pods) {
  const prompt = document.getElementById('setup-prompt');
  if (!prompt) return;

  const paycheckDone = takeHome > 0;
  const billsDone = bills > 0;
  const podsDone = pods > 0;

  if (paycheckDone && billsDone && podsDone) {
    prompt.style.display = 'none';
    return;
  }

  updateCheckItem('setup-paycheck', paycheckDone);
  updateCheckItem('setup-bills', billsDone);
```

```
  updateCheckItem('setup-pods', podsDone);
}

function updateCheckItem(id, done) {
  const el = document.getElementById(id);
  if (!el) return;
  const check = el.querySelector('.setup-check');
  if (done) {
    check.textContent = '✅';
    el.classList.add('done');
  } else {
    check.textContent = '⬜';
    el.classList.remove('done');
  }
}

function updatePaydayCountdown() {
  const el = document.getElementById('payday-countdown');
  if (!el || !appState.paycheck.frequency) return;

  const today = new Date();
  const dayOfWeek = today.getDay();
  const freq = appState.paycheck.frequency;

  let daysUntil = 0;

  if (freq === 'weekly') {
    daysUntil = (5 - dayOfWeek + 7) % 7 || 7;
  } else if (freq === 'biweekly') {
    daysUntil = (5 - dayOfWeek + 7) % 7 || 14;
  } else if (freq === 'semimonthly') {
    const day = today.getDate();
    daysUntil = day < 15 ? 15 - day : 30 - day;
  } else {
    const day = today.getDate();
    daysUntil = 30 - day;
  }

  const icon = daysUntil <= 3 ? '🎉' : '📅';
  const msg = daysUntil === 0
    ? "It's PAYDAY! 🎉 Time to check your numbers!"
    : daysUntil === 1
    ? "Payday is TOMORROW! 🤑"
    : `Next payday in ${daysUntil} days`;

  el.querySelector('.countdown-icon').textContent = icon;
  el.querySelector('.countdown-text').textContent = msg;
```

```
}

// ============================================
// BILLS
// ============================================

function showAddBill() {
  openModal('Add a Bill 📋', `
    <div class="form-group">
      <label class="form-label">Bill Name</label>
      <input type="text" id="bill-name" class="input-field" placeholder="e.g. Ren
    </div>
    <div class="form-group">
      <label class="form-label">Amount</label>
      <div class="input-prefix-wrap">
        <span class="input-prefix">$</span>
        <input type="number" id="bill-amount" class="input-field prefix" placehol
      </div>
    </div>
    <div class="form-group">
      <label class="form-label">Due Date (day of month)</label>
      <input type="number" id="bill-due" class="input-field" placeholder="e.g. 15
    </div>
    <div class="form-group">
      <label class="form-label">Category</label>
      <select id="bill-category" class="input-field select">
        <option value="🏠">🏠 Housing</option>
        <option value="⚡">⚡ Utilities</option>
        <option value="📱">📱 Phone/Internet</option>
        <option value="🚗">🚗 Transportation</option>
        <option value="🎬">🎬 Entertainment</option>
        <option value="🏥">🏥 Health</option>
        <option value="📚">📚 Education</option>
        <option value="💳">💳 Credit Card</option>
        <option value="📋">📋 Other</option>
      </select>
    </div>
    <div class="form-group">
      <label class="form-label" style="display:flex;align-items:center;gap:10px">
        <input type="checkbox" id="bill-recurring" checked style="width:18px;heig
        Repeats every month
      </label>
    </div>
    <div class="modal-actions">
      <button class="btn-secondary" onclick="closeModal()">Cancel</button>
      <button class="btn-primary" onclick="saveBill()" style="max-width:none;flex
    </div>
```

```
  `);
}

function saveBill() {
  const name = document.getElementById('bill-name')?.value.trim();
  const amount = parseFloat(document.getElementById('bill-amount')?.value);
  const due = parseInt(document.getElementById('bill-due')?.value);
  const category = document.getElementById('bill-category')?.value;
  const recurring = document.getElementById('bill-recurring')?.checked;

  if (!name || !amount || !due) {
    shakeElement(document.querySelector('.modal'));
    return;
  }

  const bill = {
    id: Date.now(),
    name, amount, due, category, recurring,
    paid: false
  };

  appState.bills.push(bill);
  saveData();
  closeModal();
  renderBills();
  updateHomeScreen();
  triggerHaptic();
}

function toggleBillPaid(id) {
  const bill = appState.bills.find(b => b.id === id);
  if (!bill) return;

  bill.paid = !bill.paid;
  saveData();

  if (bill.paid) {
    launchConfetti();
    triggerHaptic('success');
  }

  renderBills();
  updateHomeScreen();
}

function renderBills() {
  const bills = appState.bills;
```

```
  const emptyEl = document.getElementById('bills-empty');
  const listEl = document.getElementById('bills-list');

  if (!emptyEl || !listEl) return;

  if (bills.length === 0) {
    emptyEl.style.display = 'flex';
    listEl.style.display = 'none';
    return;
  }

  emptyEl.style.display = 'none';
  listEl.style.display = 'block';

  const unpaid = bills.filter(b => !b.paid).sort((a, b) => a.due - b.due);
  const paid = bills.filter(b => b.paid);

  document.getElementById('unpaid-bills').innerHTML = unpaid.map(bill => billCard
  document.getElementById('paid-bills').innerHTML = paid.map(bill => billCardHTML

  // Update summary
  const total = bills.reduce((s, b) => s + b.amount, 0);
  const paidTotal = paid.reduce((s, b) => s + b.amount, 0);
  const remaining = unpaid.reduce((s, b) => s + b.amount, 0);

  animateNumber('bills-total', total, true);
  animateNumber('bills-paid', paidTotal, true);
  animateNumber('bills-remaining', remaining, true);
}

function billCardHTML(bill) {
  const today = new Date().getDate();
  const daysUntil = bill.due - today;
  const isUrgent = daysUntil >= 0 && daysUntil <= 3 && !bill.paid;
  const dueText = bill.paid ? 'Paid ✅' :
    daysUntil === 0 ? 'Due TODAY!' :
    daysUntil === 1 ? 'Due TOMORROW!' :
    daysUntil < 0 ? `Was due on the ${bill.due}${ordinal(bill.due)}` :
    `Due in ${daysUntil} days (${bill.due}${ordinal(bill.due)})`;

  return `
    <div class="bill-card ${bill.paid ? 'paid' : ''}">
      <div class="bill-icon">${bill.category}</div>
      <div class="bill-info">
        <div class="bill-name">${bill.name}</div>
        <div class="bill-due ${isUrgent ? 'urgent' : ''}">${dueText}</div>
      </div>
```

```
      <div class="bill-amount">$${bill.amount.toFixed(2)}</div>
      <button class="bill-toggle ${bill.paid ? 'paid' : ''}" onclick="toggleBillP
        ${bill.paid ? '✓' : ''}
      </button>
    </div>
  `;
}

function ordinal(n) {
  const s = ['th', 'st', 'nd', 'rd'];
  const v = n % 100;
  return s[(v - 20) % 10] || s[v] || s[0];
}

// ============================================
// RECEIPT TRACKER
// ============================================

function showAddReceipt() {
  openModal('Log a Purchase 🧾', `
    <div class="form-group">
      <label class="form-label">Store / Place</label>
      <input type="text" id="receipt-store" class="input-field" placeholder="e.g.
    </div>
    <div class="form-group">
      <label class="form-label">Amount</label>
      <div class="input-prefix-wrap">
        <span class="input-prefix">$</span>
        <input type="number" id="receipt-amount" class="input-field prefix" place
      </div>
    </div>
    <div class="form-group">
      <label class="form-label">Category</label>
      <select id="receipt-category" class="input-field select">
        <option value="🛒 Food & Groceries">🛒 Food & Groceries</option>
        <option value="⛽ Gas">⛽ Gas</option>
        <option value="🍔 Eating Out">🍔 Eating Out</option>
        <option value="🛍️ Shopping">🛍️ Shopping</option>
        <option value="🎬 Entertainment">🎬 Entertainment</option>
        <option value="🏥 Medical">🏥 Medical</option>
        <option value="✈️ Travel">✈️ Travel</option>
        <option value="📦 Other">📦 Other</option>
      </select>
    </div>
    <div class="form-group">
      <label class="form-label">Notes (optional)</label>
      <input type="text" id="receipt-notes" class="input-field" placeholder="Any
```

```
    </div>
    <div class="modal-actions">
      <button class="btn-secondary" onclick="closeModal()">Cancel</button>
      <button class="btn-primary" onclick="saveReceipt()" style="max-width:none;f
    </div>
  `);
}

function saveReceipt() {
  const store = document.getElementById('receipt-store')?.value.trim();
  const amount = parseFloat(document.getElementById('receipt-amount')?.value);
  const category = document.getElementById('receipt-category')?.value;
  const notes = document.getElementById('receipt-notes')?.value.trim();

  if (!store || !amount) {
    shakeElement(document.querySelector('.modal'));
    return;
  }

  const receipt = {
    id: Date.now(),
    store, amount, category, notes,
    date: new Date().toLocaleDateString()
  };

  appState.receipts.push(receipt);
  saveData();
  closeModal();
  renderReceipts();
  triggerHaptic();
}

function renderReceipts() {
  const container = document.getElementById('spending-categories');
  const empty = document.getElementById('receipts-empty');
  if (!container || !empty) return;

  if (appState.receipts.length === 0) {
    empty.style.display = 'flex';
    container.style.display = 'none';
    return;
  }

  empty.style.display = 'none';
  container.style.display = 'block';

  // Group by category
```

```
  const grouped = {};
  appState.receipts.forEach(r => {
    if (!grouped[r.category]) grouped[r.category] = 0;
    grouped[r.category] += r.amount;
  });

  const takeHome = appState.paycheck.takeHome || 0;
  const budgetPerCategory = takeHome * 0.15; // Buddy suggestion: 15% per categor

  container.innerHTML = Object.entries(grouped).map(([cat, total]) => {
    const pct = budgetPerCategory > 0 ? Math.min((total / budgetPerCategory) * 10
    const color = pct < 60 ? 'green' : pct < 85 ? 'yellow' : 'red';
    return `
      <div class="category-card">
        <div class="category-header">
          <div class="category-name">${cat}</div>
          <div class="category-amount" style="color:var(--${color === 'green' ? '
            $${total.toFixed(2)}
          </div>
        </div>
        <div class="progress-bar-wrap">
          <div class="progress-bar-fill ${color}" style="width:${pct}%"></div>
        </div>
      </div>
    `;
  }).join('');
}

// ============================================
// SAVING PODS
// ============================================

function showAddPod() {
  if (appState.pods.length >= 5) {
    openModal('Max Pods Reached 🫙', `
      <div class="empty-state" style="padding:20px">
        <img src="images/buddy-confused.png" class="empty-buddy" onerror="this.st
        <p class="empty-text">You can have up to 5 saving pods at a time. Complet
      </div>
    `);
    return;
  }

  const suggested = appState.paycheck.takeHome > 0
    ? (appState.paycheck.takeHome * 0.1).toFixed(2)
    : '';
```

```
  openModal('Create a Saving Pod 🫙', `
    <div class="form-group">
      <label class="form-label">What are you saving for?</label>
      <input type="text" id="pod-name" class="input-field" placeholder="e.g. Vaca
    </div>
    <div class="form-group">
      <label class="form-label">Goal Amount</label>
      <div class="input-prefix-wrap">
        <span class="input-prefix">$</span>
        <input type="number" id="pod-target" class="input-field prefix" placehold
      </div>
    </div>
    <div class="form-group">
      <label class="form-label">Already Saved</label>
      <div class="input-prefix-wrap">
        <span class="input-prefix">$</span>
        <input type="number" id="pod-current" class="input-field prefix" placehol
      </div>
    </div>
    <div class="form-group">
      <label class="form-label">Save per Paycheck</label>
      <div class="input-prefix-wrap">
        <span class="input-prefix">$</span>
        <input type="number" id="pod-contribution" class="input-field prefix" pla
      </div>
      ${suggested ? `<p style="font-size:0.8rem;color:var(--blue);font-weight:700
    </div>
    <div class="modal-actions">
      <button class="btn-secondary" onclick="closeModal()">Cancel</button>
      <button class="btn-primary" onclick="savePod()" style="max-width:none;flex:
    </div>
  `);
}

function savePod() {
  const name = document.getElementById('pod-name')?.value.trim();
  const target = parseFloat(document.getElementById('pod-target')?.value);
  const current = parseFloat(document.getElementById('pod-current')?.value) || 0;
  const contribution = parseFloat(document.getElementById('pod-contribution')?.va

  if (!name || !target) {
    shakeElement(document.querySelector('.modal'));
    return;
  }

  const pod = {
    id: Date.now(),
```

```
    name, target, current, contribution,
    emoji: getPodEmoji(name)
  };

  appState.pods.push(pod);
  saveData();
  closeModal();
  renderPods();
  updateHomeScreen();
  triggerHaptic();
}

function getPodEmoji(name) {
  const lower = name.toLowerCase();
  if (lower.includes('vacation') || lower.includes('trip') || lower.includes('tra
  if (lower.includes('emergency') || lower.includes('fund')) return '🛡️';
  if (lower.includes('car') || lower.includes('vehicle')) return '🚗';
  if (lower.includes('home') || lower.includes('house')) return '🏠';
  if (lower.includes('wedding')) return '💍';
  if (lower.includes('phone')) return '📱';
  if (lower.includes('school') || lower.includes('education')) return '📚';
  if (lower.includes('christmas') || lower.includes('holiday')) return '🎄';
  return '🫙';
}

function renderPods() {
  const emptyEl = document.getElementById('pods-empty');
  const listEl = document.getElementById('pods-list');
  const addBtn = document.getElementById('add-pod-btn');

  if (!emptyEl || !listEl) return;

  if (appState.pods.length === 0) {
    emptyEl.style.display = 'flex';
    listEl.style.display = 'none';
    return;
  }

  emptyEl.style.display = 'none';
  listEl.style.display = 'block';

  if (addBtn) {
    addBtn.style.display = appState.pods.length >= 5 ? 'none' : 'block';
  }

  const periods = PAY_PERIODS[appState.paycheck.frequency || 'biweekly'];
```

```
  listEl.innerHTML = appState.pods.map(pod => {
    const pct = Math.min((pod.current / pod.target) * 100, 100);
    const remaining = pod.target - pod.current;
    const paychecksLeft = pod.contribution > 0 ? Math.ceil(remaining / pod.contri
    const monthsLeft = pod.contribution > 0
      ? Math.ceil(remaining / (pod.contribution * (periods / 12)))
      : '?';

    return `
      <div class="pod-card" onclick="showPodDetail(${pod.id})">
        <div class="pod-header">
          <div class="pod-name">${pod.emoji} ${pod.name}</div>
          <div class="pod-amounts">$${pod.current.toFixed(0)} / $${pod.target.toF
        </div>
        <div class="pod-progress-wrap">
          <div class="pod-progress-fill" style="width:${pct}%"></div>
        </div>
        <div class="pod-footer">
          <span class="pod-percent">${pct.toFixed(0)}% there!</span>
          <span>~${monthsLeft} months left</span>
        </div>
      </div>
    `;
  }).join('');
}

function showPodDetail(id) {
  const pod = appState.pods.find(p => p.id === id);
  if (!pod) return;

  const pct = Math.min((pod.current / pod.target) * 100, 100);
  const remaining = pod.target - pod.current;

  openModal(`${pod.emoji} ${pod.name}`, `
    <div style="text-align:center;margin-bottom:20px">
      <div style="font-family:var(--font-mono);font-size:2rem;font-weight:700;col
        $${pod.current.toFixed(2)}
      </div>
      <div style="color:var(--text-mid);font-size:0.85rem;font-weight:600">of $${
      <div style="margin:16px 0;height:16px;background:var(--blue-bg);border-radi
        <div style="height:100%;width:${pct}%;background:linear-gradient(90deg,va
      </div>
      <div style="font-size:1.2rem;font-weight:800;color:var(--blue)">${pct.toFix
    </div>

    <div class="form-group">
      <label class="form-label">Add to this pod</label>
```

```
      <div class="input-prefix-wrap">
        <span class="input-prefix">$</span>
        <input type="number" id="pod-add-amount" class="input-field prefix" place
      </div>
    </div>
    <div class="modal-actions">
      <button class="btn-secondary" onclick="deletePod(${pod.id})">Delete 🗑️</but
      <button class="btn-primary" onclick="addToPod(${pod.id})" style="max-width:
    </div>
  `);
}

function addToPod(id) {
  const amount = parseFloat(document.getElementById('pod-add-amount')?.value);
  if (!amount || amount <= 0) return;

  const pod = appState.pods.find(p => p.id === id);
  if (!pod) return;

  pod.current = Math.min(pod.current + amount, pod.target);
  saveData();
  closeModal();
  renderPods();
  triggerHaptic();

  if (pod.current >= pod.target) {
    launchConfetti();
    setTimeout(() => {
      openModal('Goal Reached! 🎉', `
        <div style="text-align:center;padding:20px">
          <img src="images/buddy-hype.png" style="width:120px;height:120px;object
          <h2 style="font-family:var(--font-display);font-size:1.8rem;color:var(-
          <p style="color:var(--text-mid);font-size:1rem;font-weight:600">Your ${
        </div>
      `);
    }, 500);
  }
}

function deletePod(id) {
  appState.pods = appState.pods.filter(p => p.id !== id);
  saveData();
  closeModal();
  renderPods();
  updateHomeScreen();
}
```

```
// ============================================
// SETTINGS
// ============================================

function updateSettingsScreen() {
  const nameEl = document.getElementById('settings-name');
  const modeEl = document.getElementById('settings-mode');

  if (nameEl) nameEl.textContent = appState.user.name || 'Hey there!';
  if (modeEl) modeEl.textContent = appState.user.mode === 'household' ? '👪 House

  const darkToggle = document.getElementById('dark-mode-toggle');
  if (darkToggle) darkToggle.checked = appState.settings.darkMode;

  const tipsToggle = document.getElementById('tips-toggle');
  if (tipsToggle) tipsToggle.checked = appState.settings.tipsEnabled;

  const quietToggle = document.getElementById('quiet-toggle');
  if (quietToggle) quietToggle.checked = appState.settings.quietMode;
}

function toggleDarkMode() {
  appState.settings.darkMode = !appState.settings.darkMode;
  document.body.classList.toggle('dark', appState.settings.darkMode);
  saveData();
}

function toggleTips() {
  appState.settings.tipsEnabled = !appState.settings.tipsEnabled;
  saveData();
}

function toggleQuiet() {
  appState.settings.quietMode = !appState.settings.quietMode;
  saveData();
}

function exportData() {
  const data = JSON.stringify(appState, null, 2);
  const blob = new Blob([data], { type: 'application/json' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = `paycheck-buddy-backup-${new Date().toISOString().split('T')[0]}.j
  a.click();
  URL.revokeObjectURL(url);
```

```
  openModal('Backup Saved! 💾', `
    <div style="text-align:center;padding:20px">
      <img src="images/buddy-happy.png" style="width:100px;height:100px;object-fi
      <p style="margin-top:16px;font-size:1rem;font-weight:600;color:var(--text-m
    </div>
  `);
}

function importData() {
  const input = document.createElement('input');
  input.type = 'file';
  input.accept = '.json';
  input.onchange = (e) => {
    const file = e.target.files[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onload = (ev) => {
      try {
        const data = JSON.parse(ev.target.result);
        appState = { ...appState, ...data };
        saveData();
        updateHomeScreen();
        updateSettingsScreen();
        renderBills();
        renderPods();
        openModal('Data Restored! ✅', `
          <div style="text-align:center;padding:20px">
            <img src="images/buddy-happy.png" style="width:100px;height:100px;obj
            <p style="margin-top:16px;font-size:1rem;font-weight:600;color:var(--
          </div>
        `);
      } catch {
        openModal('Oops! ❌', `
          <div style="text-align:center;padding:20px">
            <img src="images/buddy-confused.png" style="width:100px;height:100px;
            <p style="margin-top:16px;font-size:1rem;font-weight:600;color:var(--
          </div>
        `);
      }
    };
    reader.readAsText(file);
  };
  input.click();
}

function confirmReset() {
  openModal('Reset App? 🗑️', `
```

```
    <div style="text-align:center;padding:20px">
      <img src="images/buddy-sad.png" style="width:100px;height:100px;object-fit:
      <p style="margin:16px 0;font-size:1rem;font-weight:600;color:var(--text-mid
      <div class="modal-actions">
        <button class="btn-secondary" onclick="closeModal()">Never Mind</button>
        <button class="btn-primary" onclick="resetApp()" style="max-width:none;fl
      </div>
    </div>
  `);
}

function resetApp() {
  localStorage.removeItem('paycheckBuddyData');
  location.reload();
}

function showQR() {
  const url = window.location.href;
  openModal('Share Paycheck Buddy 📱', `
    <div style="text-align:center;padding:20px">
      <div style="background:white;padding:16px;border-radius:16px;display:inline
        <img src="https://api.qrserver.com/v1/create-qr-code/?size=200x200&data=$
             alt="QR Code" style="width:200px;height:200px;display:block" />
      </div>
      <p style="font-size:0.85rem;color:var(--text-mid);font-weight:600;word-brea
      <button class="btn-primary" style="margin-top:16px" onclick="copyLink()">Co
    </div>
  `);
}

function copyLink() {
  navigator.clipboard?.writeText(window.location.href);
  triggerHaptic('success');
}

function installApp() {
  openModal('Install Paycheck Buddy 📲', `
    <div style="padding:10px">
      <div style="display:flex;align-items:flex-start;gap:12px;margin-bottom:16px
        <div style="font-size:1.5rem">📱</div>
        <div>
          <div style="font-weight:800;margin-bottom:4px">On iPhone (Safari)</div>
          <div style="font-size:0.85rem;color:var(--text-mid);font-weight:600">Ta
        </div>
      </div>
      <div style="display:flex;align-items:flex-start;gap:12px">
        <div style="font-size:1.5rem">🤖</div>
```

```
        <div>
          <div style="font-weight:800;margin-bottom:4px">On Android (Chrome)</div
          <div style="font-size:0.85rem;color:var(--text-mid);font-weight:600">Ta
        </div>
      </div>
    </div>
  `);
}

function editProfile() {
  openModal('Edit Profile 👤', `
    <div class="form-group">
      <label class="form-label">Your Name</label>
      <input type="text" id="edit-name" class="input-field" value="${appState.use
    </div>
    <div class="form-group">
      <label class="form-label">Account Type</label>
      <select id="edit-mode" class="input-field select">
        <option value="solo" ${appState.user.mode === 'solo' ? 'selected' : ''}>🧍
        <option value="household" ${appState.user.mode === 'household' ? 'selecte
      </select>
    </div>
    <div class="modal-actions">
      <button class="btn-secondary" onclick="closeModal()">Cancel</button>
      <button class="btn-primary" onclick="saveProfile()" style="max-width:none;f
    </div>
  `);
}

function saveProfile() {
  const name = document.getElementById('edit-name')?.value.trim();
  const mode = document.getElementById('edit-mode')?.value;
  if (!name) return;
  appState.user.name = name;
  appState.user.mode = mode;
  saveData();
  closeModal();
  updateHomeScreen();
  updateSettingsScreen();
}

function editPayPeriod() {
  switchTab('paycheck');
  closeModal();
}

function showCalendar() {
```

```
  const bills = appState.bills.filter(b => !b.paid).sort((a, b) => a.due - b.due)
  const today = new Date().getDate();

  openModal('📅 Upcoming', `
    <div style="padding:10px">
      ${bills.length === 0
        ? '<p style="text-align:center;color:var(--text-mid);font-weight:600;padd
        : bills.map(b => {
          const days = b.due - today;
          const urgency = days <= 3 ? 'var(--red)' : days <= 7 ? 'var(--yellow)'
          return `
            <div style="display:flex;align-items:center;gap:12px;padding:12px;bac
              <div style="font-size:1.4rem">${b.category}</div>
              <div style="flex:1">
                <div style="font-weight:800;font-size:0.95rem">${b.name}</div>
                <div style="font-size:0.78rem;color:${urgency};font-weight:700">
                  ${days < 0 ? 'Overdue!' : days === 0 ? 'Due TODAY!' : `Due in $
                </div>
              </div>
              <div style="font-family:var(--font-mono);font-weight:700">$${b.amou
            </div>
          `;
        }).join('')
      }
    </div>
  `);
}

// ============================================
// MODAL SYSTEM
// ============================================

function openModal(title, content) {
  document.getElementById('modal-title').textContent = title;
  document.getElementById('modal-body').innerHTML = content;
  document.getElementById('modal-overlay').style.display = 'flex';
  document.body.style.overflow = 'hidden';
}

function closeModal() {
  const overlay = document.getElementById('modal-overlay');
  overlay.style.opacity = '0';
  overlay.style.transition = 'opacity 0.2s';
  setTimeout(() => {
    overlay.style.display = 'none';
    overlay.style.opacity = '1';
    document.body.style.overflow = '';
```

```
  }, 200);
}

// ============================================
// ANIMATIONS & EFFECTS
// ============================================

function animateNumber(elementId, targetValue, isCurrency = false) {
  const el = document.getElementById(elementId);
  if (!el) return;

  const start = 0;
  const duration = 800;
  const startTime = performance.now();

  function update(currentTime) {
    const elapsed = currentTime - startTime;
    const progress = Math.min(elapsed / duration, 1);
    const eased = 1 - Math.pow(1 - progress, 3);
    const current = start + (targetValue - start) * eased;

    el.textContent = isCurrency
      ? `$${current.toFixed(2)}`
      : Math.round(current).toString();

    if (progress < 1) {
      requestAnimationFrame(update);
    }
  }

  requestAnimationFrame(update);
}

function launchConfetti() {
  const container = document.getElementById('confetti-container');
  if (!container) return;

  const colors = ['#7BC64A', '#4A90D9', '#F5C842', '#FF6B6B', '#A78BFA'];

  for (let i = 0; i < 60; i++) {
    const piece = document.createElement('div');
    piece.className = 'confetti-piece';
    piece.style.left = `${Math.random() * 100}%`;
    piece.style.top = '-10px';
    piece.style.background = colors[Math.floor(Math.random() * colors.length)];
    piece.style.width = `${6 + Math.random() * 8}px`;
    piece.style.height = `${6 + Math.random() * 8}px`;
```

```
    piece.style.borderRadius = Math.random() > 0.5 ? '50%' : '2px';
    piece.style.animationDuration = `${1.5 + Math.random() * 2}s`;
    piece.style.animationDelay = `${Math.random() * 0.5}s`;
    container.appendChild(piece);

    setTimeout(() => piece.remove(), 3000);
  }
}

function shakeElement(el) {
  if (!el) return;
  el.classList.remove('shake');
  void el.offsetWidth;
  el.classList.add('shake');
  setTimeout(() => el.classList.remove('shake'), 500);
  triggerHaptic('error');
}

function triggerHaptic(type = 'light') {
  if (!navigator.vibrate) return;
  switch (type) {
    case 'light': navigator.vibrate(10); break;
    case 'success': navigator.vibrate([10, 50, 10]); break;
    case 'error': navigator.vibrate([50, 30, 50]); break;
    default: navigator.vibrate(10);
  }
}

// ============================================
// DATA PERSISTENCE
// ============================================

function saveData() {
  try {
    localStorage.setItem('paycheckBuddyData', JSON.stringify(appState));
  } catch (e) {
    console.warn('Could not save data:', e);
  }
}

function loadData() {
  try {
    const saved = localStorage.getItem('paycheckBuddyData');
    if (saved) {
      const parsed = JSON.parse(saved);
      appState = { ...appState, ...parsed };
```

```
      // Apply saved settings
      if (appState.settings.darkMode) {
        document.body.classList.add('dark');
      }
    }
  } catch (e) {
    console.warn('Could not load data:', e);
  }
}

// ============================================
// SERVICE WORKER REGISTRATION
// ============================================

if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('sw.js')
      .then(reg => console.log('SW registered'))
      .catch(err => console.log('SW registration failed:', err));
  });
}
```